

ReaLM: Residual Quantization Bridging Knowledge Graph Embeddings and Large Language Models

Wenbin Guo
wenff@tju.edu.cn
College of Intelligence and
Computing, Tianjin University
Tianjin, China

Xin Wang*
wangx@tju.edu.cn
School of Artificial Intelligence,
College of Intelligence and
Computing, Tianjin University
Tianjin, China

Jiaoyan Chen
jiaoyan.chen@manchester.ac.uk
Department of Computer Science,
The University of Manchester
Manchester, United Kingdom

Lingbing Guo
lbguo@tju.edu.cn
College of Intelligence and
Computing, Tianjin University
Tianjin, China

Zhao Li
lizh@tju.edu.cn
College of Intelligence and
Computing, Tianjin University
Tianjin, China

Zirui Chen
zrchen@tju.edu.cn
College of Intelligence and
Computing, Tianjin University
Tianjin, China

Abstract

Large Language Models (LLMs) have recently emerged as a powerful paradigm for Knowledge Graph Completion (KGC), offering strong reasoning and generalization capabilities beyond traditional embedding-based approaches. However, existing LLM-based methods often struggle to fully exploit structured semantic representations, as the continuous embedding space of pretrained KG models is fundamentally misaligned with the discrete token space of LLMs. This discrepancy hinders effective semantic transfer and limits their performance. To address this challenge, we propose ReaLM, a novel and effective framework that *bridges the gap between KG embeddings and LLM tokenization* through the mechanism of *residual vector quantization*. ReaLM discretizes pretrained KG embeddings into compact code sequences and integrates them as learnable tokens within the LLM vocabulary, enabling seamless fusion of symbolic and contextual knowledge. Furthermore, we incorporate ontology-guided class constraints to enforce semantic consistency, refining entity predictions based on class-level compatibility. Extensive experiments on two widely used benchmark datasets demonstrate that ReaLM achieves state-of-the-art performance, confirming its effectiveness in *aligning structured knowledge with large-scale language models*. The implementation is publicly available at <https://github.com/xiumu-gg/ReaLM>.

CCS Concepts

• **Computing methodologies** → **Reasoning about belief and knowledge**; *Natural language generation*; *Ontology engineering*.

Keywords

Knowledge Graph Completion, Large Language Models, Link Prediction, Quantitative Code, Semantic Representation

*Corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792437>

ACM Reference Format:

Wenbin Guo, Xin Wang, Jiaoyan Chen, Lingbing Guo, Zhao Li, and Zirui Chen. 2026. ReaLM: Residual Quantization Bridging Knowledge Graph Embeddings and Large Language Models. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792437>

1 Introduction

Knowledge Graphs (KGs) have become indispensable infrastructures for organizing and representing structured knowledge across diverse domains, including search engines [40], recommender systems [35], and biomedical informatics [10]. By encoding factual information into triples of entities and relations, KGs provide a machine-interpretable foundation for reasoning and knowledge-driven applications. Yet, real-world KGs are inherently incomplete, as many valid facts remain unobserved due to the high cost of manual curation and the evolving nature of knowledge [13]. Addressing this incompleteness through Knowledge Graph Completion (KGC), which is a task of inferring missing links, has therefore become a central research problem for enhancing the coverage and reliability of knowledge-centric systems [1].

Recent advances in large language models (LLMs) have inspired attempts to leverage their reasoning capabilities for KGC, yielding promising results in triple classification and link prediction tasks. **Nevertheless, a fundamental challenge remains: the intrinsic mismatch between continuous KG entity embedding and the discrete token space of LLMs** [43]. When a KG is directly converted into textual form and fed into an LLM, the generated results are expressed in natural language tokens rather than the exact entities defined in the KG. Consequently, these outputs cannot be directly aligned with the entity embeddings used in the original graph, leading to inconsistencies and ambiguity in entity mapping. A straightforward alternative is to incorporate each KG entity as a unique token within the LLM vocabulary, thereby enabling explicit generation of KG entities. However, this approach introduces substantial challenges. Unlike natural language vocabularies, which are fixed and relatively compact, knowledge graphs generally encompass a vast and continuously evolving collection of entities, often reaching tens or even hundreds of thousands in

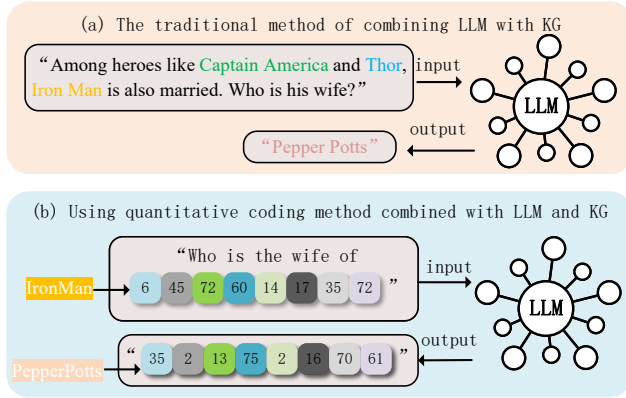


Figure 1: Replacing text-based entity representations with quantized code sequences for explicit and efficient LLM-KG integration.

number, and in some cases surpassing the token vocabulary size of standard large language models. For instance, the LLaMA-2 model employs a vocabulary of 32,000 tokens, while the WN18RR dataset alone contains 40,943 distinct entities [12]. Expanding the LLM vocabulary to include all these entities would dramatically increase the token space, leading to excessive memory consumption, slower training, and unstable inference. As a result, generative LLM-based approaches struggle to produce valid and consistent KG entities directly, limiting their effectiveness and scalability in practical KGC scenarios.

Several recent efforts have sought to bridge the gap between LLMs and KGs for completion tasks. For example, MKGL converts KG triples into textual corpora and fine-tunes LLMs to directly generate entities, but this approach often yields semantically related yet invalid outputs that deviate from the exact entity set [12]. KoPA adopts a different strategy by employing a prefix adapter that projects entity and relation embeddings into the same dimensional space as tokens, enabling LLMs to perform triple classification. However, such a direct dimensional transformation neither supports ranking-based tasks nor fully captures the rich semantics inherent to KGs [43]. More recently, SSQR introduces a novel quantization-based approach: it reconstructs structural information and employs semantic distillation losses to map entity embeddings into a discrete codebook, which is then partially integrated into the LLM tokenizer [25]. While this method achieves competitive performance in KGC, it requires the careful joint optimization of structural reconstruction, semantic distillation, and quantization objectives, which is an intricate process highly prone to imbalance during training.

To address these limitations, we propose **ReaLM**, a unified framework that seamlessly integrates **semantics of pretrained KG embeddings** into LLMs while preserving both **semantic fidelity and computational efficiency**. As illustrated in Figure 1, traditional approaches typically convert triples into textual descriptions and rely on the LLM to generate entity names in natural language. However, such text-based formulations often fail to align generated tokens with the **discrete entities defined in the KG**. In

contrast, ReaLM adopts a **quantitative coding paradigm**, where KG entities are represented by sequences of compact discrete codes derived through **residual vector quantization (RVQ)**. This encoding strategy preserves the **relational structure and semantic nuances** of pretrained embeddings while ensuring compatibility with the limited token space of LLM, thereby avoiding uncontrolled vocabulary expansion. The quantized codes are incorporated as new tokens within the LLM vocabulary and jointly optimized with its internal parameters, enabling the model to **internalize KG semantics without sacrificing linguistic fluency**. Furthermore, ReaLM integrates **ontology-guided class constraints**, ensuring that entity predictions remain consistent with their corresponding classes, which enhances both **semantic correctness**. The integration of residual quantization with ontology-based refinement enables ReaLM to jointly address semantic misalignment, vocabulary scalability, and entity-level controllability.

To illustrate, consider predicting the missing tail entity in the triple (*Ironman*, *hasWife*, ?). ReaLM encodes *Ironman* into a pretrained RotatE embedding, which is discretized via RVQ into a sequence of codebook indices (e.g., [7][48][109]...[55]) corresponding to LLM tokens. Conditioned on these tokens and the relation *hasWife*, the LLM generates a candidate entity, which is decoded through the entity-codebook mapping under ontology-guided class constraints, yielding *PepperPotts*. Our main contributions are summarized as follows:

- We introduce a residual vector quantization framework that transforms high-dimensional continuous KG embeddings into compact discrete code sequences, enabling seamless integration with LLM token spaces while preserving rich relational and semantic information.
- We incorporate ontology-based class constraints to enforce semantic consistency between predicted entities and their corresponding classes, thereby enhancing both accuracy in KGC.
- We demonstrate the effectiveness of our approach through extensive experiments on benchmark datasets, including FB15K237 and WN18RR, where ReaLM significantly outperforms existing LLM-based and traditional KG completion methods.

2 Related Work

2.1 Traditional Embedding-based KGC Models

Traditional KGC models typically adopt geometric embedding approaches, assuming relational consistency, where relations imply specific transformations that align entity vectors [18]. Early methods, such as TransE [1], treat relations as translations between entity embeddings, while subsequent models enhance representational capacity through more sophisticated operator spaces, including hyperplanes [36], relational mapping spaces [16], Riemannian spheres [26], multi-mapping spaces [7], rotational spaces [30], and convolutional spaces [5, 20, 27]. The integration of attention mechanisms and deep architectures has markedly advanced KGC, allowing models to capture intricate relational dependencies and contextual patterns. Techniques such as relation-aware attention in RelAtt [29], path-aggregation strategies in HRAN [24], and Transformer-based

frameworks like KGT5 and HITTER [3, 28] enable scalable, context-sensitive representation learning, effectively modeling higher-order interactions within large-scale KGs. Nevertheless, embedding-based approaches remain inherently limited, as they often fail to distinguish semantically similar yet class-inconsistent entities, that is, entities that are close in the embedding space but belong to different ontological classes within the knowledge graph. This limitation frequently results in semantically plausible but class-inconsistent predictions and weak generalization to unseen entities [8].

2.2 Quantization-based and Parameter-efficient KGC Models

Quantization in KG embedding is a pivotal technique that involves the transformation of high-dimensional continuous entity representations into a discrete or compact code space. This process is primarily aimed at reducing the memory footprint and computational cost associated with traditional embedding methods, while retaining the essential semantic information inherent in the entities and their relationships [23, 44]. Recent models have leveraged this idea to scale embeddings to large graphs efficiently. NodePiece [9] constructs a fixed-size entity vocabulary by selecting a subset of entities as anchors and representing each entity through its distances to these anchors. EARL [2] incorporates relation information to improve anchor matching, and RandomEQ [21] simplifies the process through random selection of anchors and relations, achieving competitive results with minimal supervision.

These traditional quantization-based and parameter-efficient methods primarily focus on directly optimizing embeddings for reasoning, without considering the integration with LLMs, which could potentially enhance their performance by leveraging the contextual understanding and reasoning capabilities of LLMs.

2.3 LLM-Based KGC Models

With the rapid advancement of LLMs, increasing efforts have been devoted to bridging the gap between the structured knowledge of KGs and the generative reasoning capabilities of LLMs. Pre-trained model approaches such as KG-BERT [41] reformulate triples into textual sequences and fine-tune BERT to classify their plausibility. While effective, they are constrained by their encoder-only architecture and struggle to seamlessly integrate the implicit knowledge of Pre-trained models with the explicit semantics of KGs [39]. In contrast, recent LLM-based KGC models exploit the generative and reasoning strengths of decoder-based architectures, either by converting structured KG information into textual inputs—often via zero-shot reasoning or instruction tuning—or by aligning KG embeddings with the token space of LLMs for tighter integration. Representative examples include MuKDC, which prompts LLMs with subgraph-based questions to infer missing triples [22], and KLR-KGC, which enhances reasoning through analogical and subgraph retrieval [17]. Other methods, such as TEA-GLM, treat LLMs as zero-shot learners for graph machine learning tasks [34], while KGEitor encodes KG paths as chain-of-thought prompts to support multi-hop reasoning [4]. MKGL converts KG triples into textual corpora and fine-tunes LLMs to generate entities directly, but often produces semantically related yet invalid outputs outside the predefined entity set [12]; KoPA introduces a prefix adapter to

project entity and relation embeddings into the LLM token space, enabling effective triple classification but falling short in ranking tasks and semantic fidelity [43]; and SSQR reconstructs structural information and distills semantics into a discrete codebook partially integrated into the LLM tokenizer, though its joint optimization of reconstruction, distillation, and quantization introduces training instability [25]. Overall, while LLM-based methods significantly extend the reasoning capacity of KGC beyond traditional embedding models, they remain challenged by the need to generate predictions faithful to the closed KG vocabulary, maintain semantic consistency across entity classes, and ensure stable integration of structured embeddings with discrete token-based models.

3 Preliminaries

Let $\mathcal{E}$ and $\mathcal{R}$ denote the sets of entities and relations in a KG, respectively, where each triple $(h, r, t) \in \mathcal{F} \subseteq \mathcal{E} \times \mathcal{R} \times \mathcal{E}$ represents a relation r linking head entity h to tail entity t . Each entity $e \in \mathcal{E}$ may also be associated with a class $y(e) \in \mathcal{Y}$. In the KGC task, the objective is to predict a missing entity in an incomplete triple, e.g., $(h, r, ?)$ or $(?, r, t)$, by ranking candidate entities based on a scoring function $f(h, r, t')$. When ontology knowledge is available, predictions are further constrained to ensure class consistency, accepting $\hat{t}$ only if $y(\hat{t}) = \hat{y}$.

To enhance representation efficiency, continuous embeddings $\mathbf{e}$ capturing semantic and relational information can be quantized into discrete latent codes $[i_1, \dots, i_S]$, thereby enabling compact storage and efficient computation. In LLM-based frameworks, triples are reformulated as textual sequences, allowing the model to either generate the missing entity ID or evaluate the plausibility of candidate triples, bridging symbolic reasoning in KGs with the generative capability of LLMs.

4 Methodology

RealLM integrates KG semantics into LLMs through a structured, multi-stage process. It first encodes entities into embeddings, then discretizes these embeddings via residual vector quantization to produce compact token representations. These tokens are incorporated into the LLM vocabulary for fine-tuning, while ontology-guided class information enforces semantic consistency during prediction. This design ensures that structured knowledge is preserved and effectively leveraged in the generative model.

4.1 Knowledge Graph Semantic Extraction

A key challenge in integrating KGs with LLMs lies in extracting semantic representations that are both structurally informative and readily transferable to downstream tasks. While numerous embedding methods have been proposed for KGs, such as TransE, DistMult, and ComplEx, our preliminary evaluation demonstrates that RotatE [30] consistently achieves superior performance in capturing relational patterns crucial for KGC, making it particularly suitable as the semantic backbone for our framework.

Formally, given a triple $(h, r, t) \in \mathcal{F}$, RotatE embeds entities $\mathbf{e}_h, \mathbf{e}_t \in \mathbb{C}^d$ and relation $\mathbf{e}_r \in \mathbb{C}^d$ in the complex vector space, where relations are parameterized as rotations. The scoring function is defined as:

$$f(h, r, t) = -\|\mathbf{e}_h \circ \mathbf{e}_r - \mathbf{e}_t\| \quad (1)$$

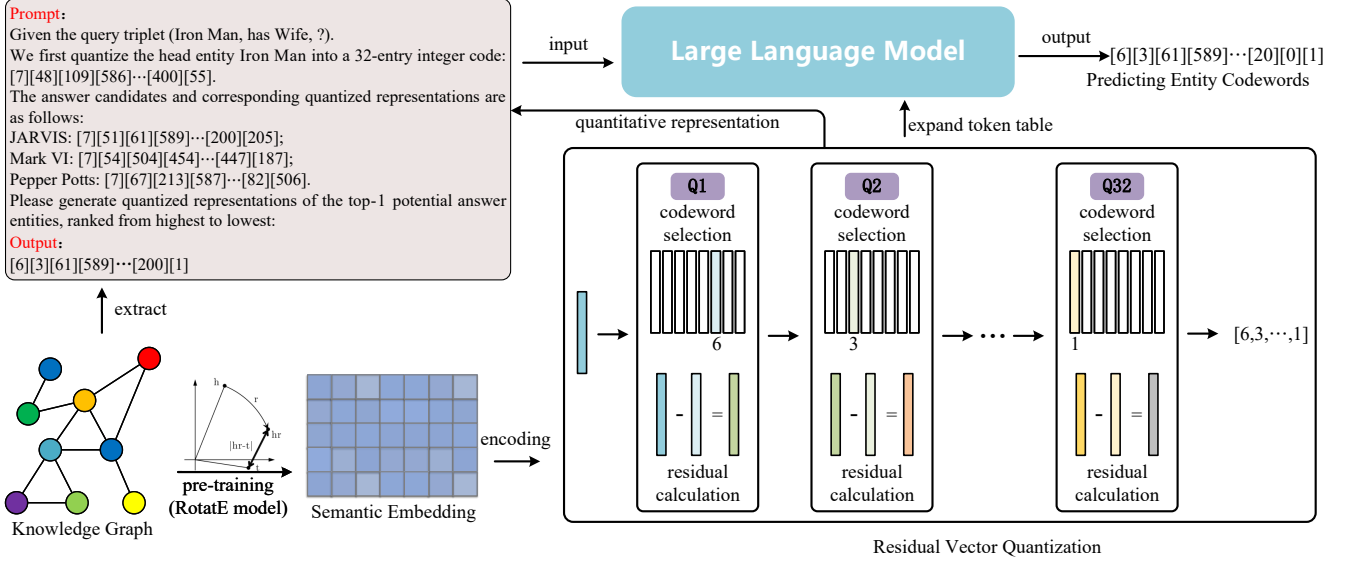


Figure 2: The overall framework of RealLM.

where d represents the dimension of embedding, and $\circ$ represents element-wise (Hadamard) multiplication.

To obtain pretrained semantic embeddings, we train RotatE on the target KG until convergence using standard negative sampling and margin-based ranking loss. The objective is:

$$\mathcal{L}_{\text{RotatE}} = - \sum_{(h,r,t) \in \mathcal{F}} \log \sigma(\gamma - f(h, r, t)) - \sum_{(h',r,t') \in \mathcal{F}'} \log \sigma(f(h', r, t') - \gamma) \quad (2)$$

where $\mathcal{F}'$ is the set of negative samples constructed by corrupting the triples, γ is a margin hyper-parameter, and $\sigma(\cdot)$ is the sigmoid function.

Through this pretraining process, we obtain high-quality entity embeddings $\{e \in \mathbb{C}^d \mid e \in \mathcal{E}\}$, which encode both structural connectivity and semantic consistency. These embeddings serve as the foundation for our residual vector quantization module, ensuring that the subsequent quantization preserves rich semantic information while bridging the gap to LLM tokenization.

4.2 Residual Vector Quantization of Semantic Embeddings

Given a pretrained entity embedding $e \in \mathbb{C}^d$ obtained from the RotatE model, our goal is to transform it into a discrete representation while minimizing information loss. To this end, we employ Residual Vector Quantization, which approximates e through a sequence of residual refinements across S stages.

Each stage $s \in \{1, \dots, S\}$ is associated with an independent codebook $C_s = \{c_{s,1}, \dots, c_{s,K}\}$, where K is the number of codewords and each $c_{s,i} \in \mathbb{R}^{2d}$. Here, the original complex-valued RotatE embedding $e \in \mathbb{C}^d$ is mapped to a real-valued vector $e_{\text{real}} \in \mathbb{R}^{2d}$ by concatenating its real and imaginary parts, so that it can be directly processed by the real-valued residual vector quantization.

At stage s , the residual vector res_{s-1} (with $\text{res}_0 = e_{\text{real}}$) is quantized by selecting the nearest codeword:

$$i = \arg \min_j \|\text{res}_{s-1} - c_{s,j}\|, \quad \text{res}_s = \text{res}_{s-1} - c_{s,i} \quad (3)$$

where i denotes the index of the selected codeword at stage s . The operator $\arg \min_j$ returns the index j that minimizes the distance between the residual vector res_{s-1} and the candidate codeword $c_{s,j}$. The quantized embedding is then given by the cumulative approximation:

$$e_q = \sum_{s=1}^S c_{s,i_s} \quad (4)$$

with its discrete representation expressed as the index sequence $[i_1, i_2, \dots, i_S]$. In our implementation, we empirically set $S = 32$, which achieves an effective trade-off between reconstruction fidelity and sequence compactness.

To optimize the codebooks and quantized representations jointly, we adopt the standard VQ objective augmented with residual refinements. The overall loss is:

$$\mathcal{L}_{\text{rc}} = \underbrace{\|e - e_q\|^2}_{\text{reconstruction loss}} + \underbrace{\sum_{s=1}^S \|\text{sg}[\text{res}_{s-1}] - c_{s,i_s}\|^2}_{\text{codebook loss}} \quad (5)$$

$$\mathcal{L}_{\text{RVQ}} = \mathcal{L}_{\text{rc}} + \underbrace{\beta \sum_{s=1}^S \|\text{res}_{s-1} - \text{sg}[c_{s,i_s}]\|^2}_{\text{commitment loss}} \quad (6)$$

where $\text{sg}[\cdot]$ denotes the stop-gradient operator and β is a weighting hyperparameter. The term $\mathcal{L}_{\text{rc}}$ combines two objectives: the reconstruction loss, which enforces fidelity between the pretrained embedding e and its quantized approximation e_q , and the codebook

loss, which moves the codewords toward the residual vectors. The additional commitment term, weighted by β , prevents codebook collapse by encouraging each residual to remain close to its selected codeword.

Through the residual vector quantization process, the pretrained semantic embedding $\mathbf{e}$ is transformed into a compact discrete sequence of S code indices. These indices not only preserve the semantic structure encoded by RotatE but also enable direct integration with LLM vocabularies via token mapping in subsequent stages.

4.3 Fine-tuning LLMs with Quantized Tokens

We extend the LLM vocabulary with K new tokens $[\text{CODE}_j]$, each corresponding to a codeword in the residual vector quantization codebooks. Each codeword embedding is obtained by summing the vectors at the same position across all S stages of the codebooks. Since the dimensionality of $\mathbf{c}_j^{\text{sum}}$ matches the LLM embedding size, we initialize the embedding of each new token with this sum, ensuring that it captures the full semantic information represented by the residual vector quantization codebooks:

$$E([\text{CODE}_j]) = \sum_{s=1}^S \mathbf{c}_{s,j} \quad (7)$$

During adaptation, two update strategies are applied in a complementary manner. First, the embeddings of the newly added tokens are trained with standard gradient descent, allowing them to refine beyond their initialization and better integrate into the LLM semantic space. The embeddings of the original vocabulary remain frozen to preserve the pretrained linguistic capacity.

Second, the internal parameters of the LLM are adapted using Low-Rank Adaptation [15], which introduces trainable low-rank matrices into the attention and feed-forward layers:

$$\mathbf{W}' = \mathbf{W} + \mathbf{A}\mathbf{B}^\top, \quad (8)$$

$$\mathbf{A} \in \mathbb{R}^{d_{\text{LLM}} \times r}, \quad \mathbf{B} \in \mathbb{R}^{k_{\text{LLM}} \times r}, \quad r \ll \min(d_{\text{LLM}}, k_{\text{LLM}})$$

where d_{LLM} and k_{LLM} denote the input and output dimensions of the weight matrix $\mathbf{W}$, respectively. LoRA introduces a low-rank adjustment $\mathbf{A}\mathbf{B}^\top$ to $\mathbf{W}$, allowing efficient fine-tuning by training only $\mathbf{A}$ and $\mathbf{B}$ while keeping $\mathbf{W}$ fixed.

This efficient adaptation enables the model to internalize the discrete quantized token format and align it with its generative process, thereby producing structured outputs that match the desired prediction format. The overall fine-tuning objective remains next-token prediction:

$$\mathcal{L}_{\text{LLM}} = \sum_{n=1}^N \log p(x_n | x_{<n}) \quad (9)$$

where x_n denotes the n -th token in the input sequence, which may consist of both natural language tokens and quantized KG tokens (the detailed construction of these mixed training sequences is described in Appendix A). Through this dual fine-tuning scheme, which combines direct optimization of new token embeddings with LoRA-based adaptation of internal parameters, the LLM learns to interpret and generate discrete knowledge graph representations while preserving its original linguistic proficiency.

4.4 Ontology-Guided Class Integration

In addition to entity-level quantization, we further integrate ontology knowledge by modeling entity classes. Each entity $e \in \mathcal{E}$ is associated with a class $y(e) \in \mathcal{Y}$, and $y(e)$ maps an entity to its corresponding class. This allows us to incorporate class-level semantic constraints into the KG completion process.

To integrate class information into our framework, we treat classes as entities and apply the same three-step procedure: (i) pre-training class embeddings with RotatE on the class graph, (ii) quantizing these embeddings using residual vector quantization, and (iii) extending the LLM vocabulary with class-specific tokens initialized from the quantized codewords. The LLM is then fine-tuned to predict missing classes in incomplete triples $(h, r, ?)$, analogous to entity prediction.

Since the number of classes is significantly smaller than the number of entities, the LLM achieves **higher prediction accuracy on classes**. We exploit this property as a semantic constraint: given an entity prediction $\hat{t}$ for a query $(h, r, ?)$, we simultaneously predict its class $\hat{y}$. If $\hat{t}$ does not belong to $\hat{y}$, we refine the prediction by selecting an alternative entity candidate t' such that $y(t') = \hat{y}$.

This ontology-guided filtering mechanism enforces **class consistency** between entity and class predictions, thereby improving robustness and semantic correctness in KG completion.

5 Experiments

In this section, we conduct a comprehensive empirical study to evaluate the proposed model ReaLM. The design of our experiments is guided by three central research questions: **RQ1**: How well does the ReaLM model perform on the standard link prediction task? **RQ2**: Can the ReaLM model be effectively extended to additional tasks such as triple classification? **RQ3**: How well does the residual vector quantization preserve semantics in the learned representations?

5.1 Experimental Setup

To systematically evaluate the performance and generalization capability of the proposed ReaLM, we conduct experiments on two representative KG reasoning tasks: *link prediction* and *triple classification*. We employ two widely used benchmark datasets, FB15k-237 [31] and WN18RR [6], along with their ontology-augmented counterparts, FB15k-237O and WN18RRO [14]. To obtain semantic embeddings of entities and relations, we adopt the RotatE model, which has demonstrated strong performance in capturing relational patterns. Both entity and relation embeddings are pretrained with a dimensionality of 2048, aligned with the token embedding size of the base LLM to facilitate seamless integration.

The semantic embeddings are discretized using residual vector quantization. We configure each codebook with 1000 codewords, and each entity is represented by a sequence of 32 code indices, balancing reconstruction fidelity and compactness. For adaptation to LLMs, we extend the vocabulary of Llama-3.2 with quantized code tokens. Fine-tuning is conducted in two complementary ways: (i) optimizing the embeddings of the newly introduced tokens, and (ii) applying parameter-efficient LoRA adaptation to the internal layers of model. The maximum training epoch is set to 100.

For link prediction, we report mean reciprocal rank and Hits@ k , which are standard metrics for evaluating ranking-based tasks. For

Table 1: KG link prediction results of our model ReaLM and the baselines including the traditional embedding-based and LLM-based models. All results of the baselines are sourced from the original papers. The best and second-best results are boldfaced and underlined, respectively.

Model	WN18RR				FB15k237			
	MRR	Hits@1	Hits@3	Hits@10	MRR	Hits@1	Hits@3	Hits@10
Embedding Methods								
TransE [1]	0.223	0.014	0.401	0.529	0.330	0.231	0.369	0.528
RotatE [30]	0.476	0.428	0.492	0.571	0.338	0.241	0.375	0.533
ConvE [5]	0.460	0.390	0.430	0.480	0.316	0.239	0.350	0.491
HyConvE [33]	0.461	0.432	-	0.534	0.339	0.212	-	0.458
HittER [3]	0.503	0.462	0.516	0.584	0.373	0.279	0.409	0.558
TCRA [11]	0.496	0.457	0.511	0.574	0.367	0.275	0.403	0.554
LLM-based Methods								
KG-BERT [41]	0.216	0.041	0.302	0.524	-	-	-	0.520
KICGPT [37]	0.549	0.474	0.585	0.641	0.412	0.327	0.448	0.554
KG-FIT [19]	0.553	0.488	0.595	<u>0.695</u>	0.362	0.275	0.485	0.572
MKGL [12]	0.552	0.500	0.577	0.656	0.415	0.325	0.454	0.591
SSQR-LLaMA2 [25]	0.591	0.548	0.618	0.673	0.449	0.374	<u>0.491</u>	<u>0.597</u>
SSQR-LLaMA3.1 [25]	<u>0.598</u>	<u>0.559</u>	0.618	0.675	<u>0.459</u>	<u>0.393</u>	<u>0.491</u>	<u>0.597</u>
ReaLM (Our Method)	0.608	0.575	<u>0.615</u>	0.699	0.467	0.402	0.498	0.603

triple classification, we adopt accuracy, precision, recall, and F1-score, providing a holistic view of classification effectiveness.

5.2 Link Prediction Analysis

For link prediction experiments, we evaluated a range of representative KG embedding methods, including TransE [1], RotatE [30], ConvE [5], HyConvE [33], HittER [3], and TCRA [11], alongside recent LLM-based approaches such as KICGPT [37], KG-FIT [19], MKGL [12], and SSQR [25]. All models were tested on the standard benchmark datasets WN18RR and FB15k237. The ontology information was extracted from WN18RR and FB15K237O datasets [14].

Result Analysis. Table 1 presents the link prediction results on WN18RR and FB15k237. Traditional embedding methods, such as TransE and ConvE, achieve moderate performance, while recent LLM-based models provide noticeable improvements due to their contextual reasoning capabilities. Our proposed ReaLM consistently outperforms all baselines across both datasets. On WN18RR, ReaLM achieves an MRR of 0.608 and Hits@10 of 0.699, improving over the strongest LLM-based baseline by 1.6–3.5%, with particularly strong gains in ranking the most relevant entities (Hits@1 of 0.575). On FB15k237, ReaLM attains an MRR of 0.467 and Hits@10 of 0.603, surpassing prior methods in all metrics and demonstrating robust generalization across datasets. Overall, these results indicate that ReaLM consistently outperforms both embedding-based and existing LLM-based approaches. This improvement can be attributed to its multi-stage design: high-quality RotatE embeddings encode rich relational structure, residual vector quantization transforms them into compact, semantically informative tokens, and the LLM is fine-tuned with LoRA to internalize these structured representations while maintaining linguistic proficiency. By bridging structured

KG knowledge and the generative capabilities of LLMs, ReaLM achieves superior link prediction performance across datasets.

Table 2: Comparison of link prediction and ablation experiment performance on WN18RR and FB15k-237 datasets.

Model	MRR	Hits@1	Hits@3	Hits@10
WN18RR				
ReaLM	0.608	0.575	0.615	0.699
Δ ($\uparrow$)	1.64%	2.78%	-0.49%	3.43%
w/o ontology	0.591	0.562	0.598	0.689
Δ ($\downarrow$)	2.80%	2.26%	2.76%	1.43%
FB15k-237				
ReaLM	0.467	0.402	0.498	0.603
Δ ($\uparrow$)	1.71%	2.24%	1.41%	1.00%
w/o ontology	0.452	0.382	0.476	0.588
Δ ($\downarrow$)	3.21%	4.98%	4.42%	2.49%

Ablation Study. To further investigate the contribution of ontology knowledge, we conducted ablation experiments in which the ontology-guided class integration component of ReaLM was removed. All other configurations, including entity embedding with RotatE, residual vector quantization, and LoRA-based LLM fine-tuning, were kept identical to the main experiments to isolate the effect of ontology information.

As shown in Table 2, the Δ symbols indicate performance changes with respect to the baseline SSQR model ($\uparrow$ for improvement, $\downarrow$ for degradation). The cells highlighted in pink denote statistically significant changes. Removing ontology knowledge leads to consistent performance degradation across both WN18RR and FB15k237. On WN18RR, MRR decreases from 0.608 to 0.591, Hits@1 drops sharply

from 0.575 to 0.562, and Hits@10 declines moderately from 0.699 to 0.689. Similarly, on FB15k237, MRR falls from 0.467 to 0.452, Hits@1 from 0.402 to 0.382, and Hits@10 from 0.603 to 0.588.

Notably, the pronounced drop in Hits@1 highlights that ontology knowledge is particularly critical for top-rank prediction accuracy, ensuring that the most relevant entities are correctly identified. The smaller reductions in Hits@10 suggest that overall ranking quality is less affected, but class-level constraints still provide a measurable boost in precision. These observations confirm that integrating ontology information enables ReaLM to maintain semantic consistency, improving both the reliability and robustness of link prediction across different datasets.

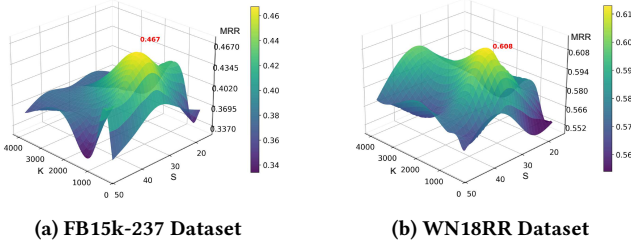


Figure 3: Impact of codebook size and quantization stages on link prediction performance

Analysis of Residual Vector Quantization Parameters. To investigate the impact of residual vector quantization hyperparameters on performance, we conducted a grid search over codebook size K and the number of quantization stages S . Figures 3(a) and (b) present 3D surface plots of link prediction MRR scores on FB15k-237 and WN18RR, respectively. Here, **FB15K-237N** denotes a label-based variant of FB15K-237 that shares the same underlying triples but stores entities using their surface names rather than numeric IDs, and is used for compatibility with text-oriented modeling.

The results indicate that both K and S significantly influence model performance. On FB15k-237, the MRR achieves its peak (0.467) at $K = 1000$, $S = 32$, suggesting that moderate codebook sizes and a sufficient number of stages are critical for capturing rich semantic information while avoiding over-compression. On WN18RR, a similar pattern emerges, with the highest MRR (0.608) observed at $K = 1500$ and $S = 32$. Performance tends to degrade when K is too small, likely due to insufficient representational capacity, or when S is excessively large, possibly introducing redundancy and overfitting in the discrete token representations. Overall, this analysis demonstrates that careful tuning of the residual vector quantization parameters enables ReaLM to produce compact yet semantically informative token sequences, thereby maximizing the effectiveness of LLM fine-tuning for link prediction tasks (answering RQ1).

5.3 Triple Classification Analysis

Result Analysis. As presented in Table 3, ReaLM achieves the highest performance among all compared models, reaching an accuracy of 0.814 and an F1-score of 0.825 on the FB15k-237N dataset. Compared with the strongest baseline SSQR-LLaMA3.1, ReaLM improves the F1-score by 1.4 percentage points, demonstrating its

superior ability to distinguish valid triples from corrupted ones. The improvement is consistent across all metrics, with gains of 1.7% in accuracy, 1.7% in precision, and 1.4% in recall. This balanced enhancement indicates that ReaLM not only identifies a wider range of correct triples but also maintains high discriminative precision. Such robustness suggests that the model effectively captures both global relational structures and local semantic constraints.

Table 3: The experiment results of the triple classification on FB15k-237N dataset. The results of baselines are taken from SSQR [25].

Model	Acc	P	R	F1
TransE [1]	0.697	0.708	0.671	0.689
ComplEx [32]	65.70	66.46	63.38	64.88
DistMult [38]	0.587	0.590	0.568	0.579
RotatE [30]	0.684	0.692	0.664	0.678
Alpaca	0.561	0.533	0.974	0.689
GPT-3.5	0.602	0.866	0.240	0.376
KG-LLaMA [42]	0.748	0.674	0.962	0.793
KG-Alpaca [42]	0.699	0.627	0.983	0.766
KoPA [43]	0.777	0.708	0.941	0.808
SSQR-LLaMA2 [25]	0.794	0.757	0.867	0.808
SSQR-LLaMA3.1 [25]	0.798	0.759	0.872	0.811
ReaLM	0.814	0.776	0.886	0.825
w/o ontology	0.783	0.754	0.856	0.798
$\Delta(\downarrow)$	3.96%	2.92%	3.50%	3.38%

The strong performance of ReaLM can be attributed to its design that integrates discrete KG representations into the token space of LLM through residual vector quantization. This alignment allows the language model to internalize entity-level semantics while retaining symbolic interpretability. Consequently, ReaLM establishes a tighter semantic correspondence between KG embeddings and textual representations, leading to more consistent reasoning and reliable triple classification.

Ablation Study on Ontology Knowledge. To examine the contribution of ontology-guided semantic constraints, we conduct an ablation study by removing the ontology integration module from ReaLM. In this setting, entities are still quantized and integrated into the LLM vocabulary, but the model no longer models class-level structures. Specifically, we omit the three-step process of pretraining, quantizing, and tokenizing class embeddings, and the LLM is trained solely to predict entities without incorporating class-level supervision. As a result, the model lacks the ontology-guided refinement mechanism that enforces semantic consistency between predicted entities and their corresponding classes.

As shown in Table 3, removing ontology knowledge leads to a consistent decline across all metrics, with decreases of 3.96% in accuracy and 3.38% in F1-score. This degradation indicates that, without class-level constraints, the model becomes more prone to semantic drift — predicting entities that are contextually plausible but type-inconsistent with the relational pattern. In other words, while the quantized entity embeddings preserve relational structure, the absence of ontology guidance weakens the ability of ReaLM to

filter out type-incoherent predictions. The results demonstrate that ontology-guided filtering plays a critical role in refining predictions of ReaLM. By aligning entity and class predictions, it effectively constrains the search space to semantically valid candidates, improving both precision and recall.

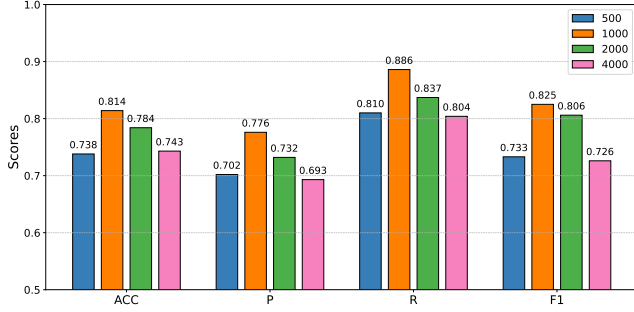


Figure 4: Impact of codebook size on triple-classification performance.

Effect of Codebook Size. We evaluate the influence of the residual vector quantization codebook size K on triple-classification performance. In our setup, each new token $[\text{CODE}_j]$ is initialized by summing its stage-wise codewords as $E([\text{CODE}_j]) = \sum_{s=1}^S \mathbf{c}_{s,j}$, where $\mathbf{c}_{s,j}$ denotes the j -th codeword in stage s . The embeddings of these added tokens are trainable while the original LLM vocabulary remains frozen, and the internal parameters of the model are adapted via LoRA. We report accuracy, precision, recall, and F1 for $K = \{500, 1000, 2000, 4000\}$ in Figure 4.

Two clear trends can be observed. Increasing K from small to moderate values consistently improves performance, as a larger codebook provides finer representational granularity and enhances the discrimination between valid and corrupted triples. However, performance gains saturate beyond an intermediate scale, with further increases in K yielding diminishing returns and occasional instability. This saturation arises from a trade-off between expressiveness and reliability. While a larger K enables finer quantization and higher recall, excessively large codebooks lead to sparsely activated codewords with insufficient training updates, which may reduce embedding reliability and slightly degrade precision.

From a practical standpoint on the FB15k-237N dataset, an intermediate codebook size offers the best trade-off between expressiveness and computational cost. The dual adaptation scheme employed in this work, which combines trainable new-token embeddings with low-rank LoRA updates, helps mitigate sparsity issues by enabling contextual integration of quantized tokens without requiring updates to the entire model. Nevertheless, overly large K still increases memory usage and the number of task-specific parameters, and requires more data or stronger regularization to avoid overfitting to rare codewords (**answering RQ2**).

5.4 Quantitative Code Analysis

To examine the impact of the codebook configuration on quantization performance, we varied the number of codewords $K \in \{4, 8, 16, 32, 64\}$ and evaluated three indicators: reconstruction mean

squared error, commitment MSE, and cosine similarity. The reconstruction and commitment MSE respectively measure how accurately the quantized embeddings approximate the original representations and how well the codewords fit the latent distribution, while cosine similarity reflects the semantic alignment between the quantized and continuous embeddings.

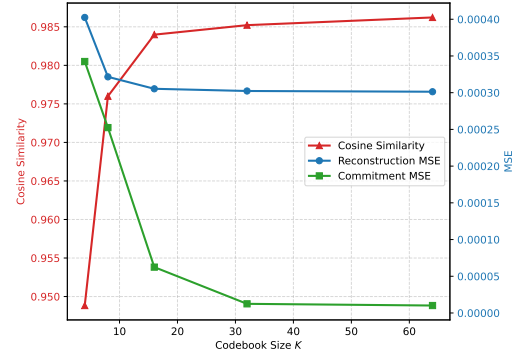


Figure 5: Impact of codebook size on triple-classification performance.

As shown in Figure 5, increasing the codebook size consistently reduces both reconstruction and commitment errors while improving cosine similarity. Specifically, the reconstruction MSE decreases from 0.000402 to 0.000301, and the commitment MSE drops by nearly two orders of magnitude, whereas the cosine similarity rises from 0.9488 to 0.9862. These results indicate that enlarging the codebook enhances the ability of model to preserve both numerical precision and semantic consistency. Overall, the proposed quantization scheme effectively compresses the KG embeddings while maintaining their relational semantics, demonstrating strong representational efficiency (**answering RQ3**).

6 Conclusion

In this paper, we proposed **ReaLM**, a unified framework that integrates knowledge graph semantics into large language models through residual vector quantization and ontology-guided refinement. By discretizing high-dimensional KG embeddings into compact code sequences, ReaLM effectively bridges the gap between continuous entity representations and the discrete token space of LLMs. Extensive experiments on standard benchmarks demonstrate that ReaLM achieves state-of-the-art performance on link prediction and triple classification tasks. Moreover, the empirical results suggest that the proposed framework is highly extensible and holds strong potential for application to broader graph-structured reasoning problems in other domains, such as protein-protein interaction analysis and job recommendation, which will be explored in future work.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (62472311), and the EPSRC Project OntoEm (EP/Y017706/1).

References

- [1] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. 2013. Translating Embeddings for Modeling Multi-Relational Data. In *Advances in Neural Information Processing Systems*. 2787–2795.
- [2] Mingyang Chen, Wen Zhang, Zhen Yao, Yushan Zhu, Yang Gao, Jeff Z. Pan, and Huajun Chen. 2023. Entity-Agnostic Representation Learning for Parameter-Efficient Knowledge Graph Embedding. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 4182–4190.
- [3] Sanxing Chen, Xiaodong Liu, Jianfeng Gao, Jian Jiao, Ruofei Zhang, and Yangfeng Ji. 2021. HittER: Hierarchical Transformers for Knowledge Graph Embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 10395–10407.
- [4] Siyuan Cheng, Ningyu Zhang, Bozhong Tian, Xi Chen, Qingbin Liu, and Huajun Chen. 2024. Editing Language Model-Based Knowledge Graph Embeddings. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 17835–17843.
- [5] Tim Dettmers, Pasquale Minervini, Pontus Stenetorp, and Sebastian Riedel. 2018. Convolutional 2D Knowledge Graph Embeddings. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*. 1811–1818.
- [6] Tim Dettmers, Pasquale Minervini, Pontus Stenetorp, and Sebastian Riedel. 2018. Convolutional 2D Knowledge Graph Embeddings. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press.
- [7] Miao Fan, Qiang Zhou, Emily Chang, and Fang Zheng. 2014. Transition-Based Knowledge Graph Embedding with Relational Mapping Properties. In *Proceedings of the 28th Pacific Asia Conference on Language, Information and Computing*. 328–337.
- [8] Tuoyu Feng, Yongsheng Wu, and Libing Li. 2024. Research on Knowledge Graph Completion Based upon Knowledge Graph Embedding. In *Proceedings of the 9th International Conference on Computer and Communication Systems*. 1335–1342.
- [9] Mikhail Galkin, Etienne Denis, Jiapeng Wu, and William L. Hamilton. 2022. NodePiece: Compositional and Parameter-Efficient Representations of Large Knowledge Graphs. In *Proceedings of the International Conference on Learning Representations*.
- [10] Yu Gu, Sue Kase, Michelle Vanni, Brian Sadler, Percy Liang, Xifeng Yan, and Yu Su. 2021. Beyond IID: Three Levels of Generalization for Question Answering on Knowledge Bases. In *Proceedings of The Web Conference 2021*. 3477–3488.
- [11] Jingtao Guo, Chunxia Zhang, Lingxi Li, Xiaojun Xue, and Zhendong Niu. 2024. A Unified Joint Approach with Topological Context Learning and Rule Augmentation for Knowledge Graph Completion. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 13686–13696.
- [12] Lingbing Guo, Zhongpu Bo, Zhuo Chen, Yichi Zhang, Jiaoyan Chen, Yarong Lan, Mengshu Sun, Zhiqiang Zhang, Yangyifei Luo, Qian Li, et al. 2024. MKGL: Mastery of a Three-Word Language. In *Advances in Neural Information Processing Systems*, Vol. 37. 140509–140534.
- [13] Wenbin Guo, Zhao Li, Xin Wang, Zirui Chen, Jun Zhao, Jianxin Li, and Ye Yuan. 2025. ConvD: Attention-Enhanced Dynamic Convolutional Embeddings for Knowledge Graph Completion. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [14] Wenbin Guo, Xin Wang, Jiaoyan Chen, Zhao Li, and Zirui Chen. 2025. Ontology-Enhanced Knowledge Graph Completion Using Large Language Models. In *Proceedings of the 24th International Semantic Web Conference*. Springer.
- [15] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *Proceedings of the International Conference on Learning Representations*.
- [16] Guoliang Ji, Shizhu He, Liheng Xu, Kang Liu, and Jun Zhao. 2015. Knowledge Graph Embedding via Dynamic Mapping Matrix. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing*. 687–696.
- [17] Shengwei Ji, Longfei Liu, Jizhong Xi, Xiaoxue Zhang, and Xinlu Li. 2024. KLR-KGC: Knowledge-Guided LLM Reasoning for Knowledge Graph Completion. *Electronics* 13, 24 (2024).
- [18] Dan Jiang, Ronggui Wang, Lixia Xue, and Juan Yang. 2024. Multisource Hierarchical Neural Network for Knowledge Graph Embedding. *Expert Systems with Applications* 237 (2024), 121446.
- [19] Pengcheng Jiang, Lang Cao, Danica Xiao, Parminder Bhatia, Jimeng Sun, and Jiawei Han. 2024. KG-FIT: Knowledge Graph Fine-Tuning upon Open-World Knowledge. *Advances in Neural Information Processing Systems* 37 (2024), 136220–136258.
- [20] Xiaotian Jiang, Quan Wang, and Bin Wang. 2019. Adaptive Convolution for Multi-Relational Learning. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 978–987.
- [21] Jiaang Li, Quan Wang, Yi Liu, Licheng Zhang, and Zhendong Mao. 2023. Random Entity Quantization for Parameter-Efficient Compositional Knowledge Graph Representation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*.
- [22] Qian Li, Zhuo Chen, Cheng Ji, Shiqi Jiang, and Jianxin Li. 2024. LLM-Based Multi-Level Knowledge Generation for Few-Shot Knowledge Graph Completion. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*.
- [23] Yunqi Li, Shuyuan Xu, Bo Liu, Zuohui Fu, Shuchang Liu, Xu Chen, and Yongfeng Zhang. 2021. Discrete Knowledge Graph Embedding Based on Discrete Optimization. *arXiv preprint arXiv:2101.XXXXX* (2021).
- [24] Zhifei Li, Hai Liu, Zhaoli Zhang, Tingting Liu, and Neal N. Xiong. 2021. Learning Knowledge Graph Embedding with Heterogeneous Relation Attention Networks. *IEEE Transactions on Neural Networks and Learning Systems* 33, 8 (2021), 3961–3973.
- [25] Qika Lin, Tianzhe Zhao, Kai He, Zhen Peng, Fangzhi Xu, Ling Huang, Jingying Ma, and Mengling Feng. 2025. Self-Supervised Quantized Representation for Seamlessly Integrating Knowledge Graphs with Large Language Models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*.
- [26] Mojtaba Nayyeri, Sahar Vahdati, Can Aykul, and Jens Lehmann. 2021. 5E: Knowledge Graph Embeddings with Projective Transformations. In *Proceedings of the AAAI Conference on Artificial Intelligence*. 9064–9072.
- [27] Dai Quoc Nguyen, Tu Dinh Nguyen, Dat Quoc Nguyen, and Dinh Phung. 2018. A Novel Embedding Model for Knowledge Base Completion Based on Convolutional Neural Network. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 327–333.
- [28] Apoorv Saxena, Adrian Kochsiek, and Rainer Gemulla. 2022. Sequence-to-Sequence Knowledge Graph Completion and Question Answering. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*. 2814–2828.
- [29] Nasrullah Sheikh, Xiao Qin, Berthold Reinwald, Christoph Miksovics, Thomas Gschwind, and Paolo Scotton. 2021. Knowledge Graph Embedding Using Graph Convolutional Networks with Relation-Aware Attention. *arXiv preprint arXiv:2102.07200* (2021).
- [30] Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. 2019. RotatE: Knowledge Graph Embedding by Relational Rotation in Complex Space. In *Proceedings of the Seventh International Conference on Learning Representations*.
- [31] Kristina Toutanova and Danqi Chen. 2015. Observed versus Latent Features for Knowledge Base and Text Inference. In *Proceedings of the 3rd Workshop on Continuous Vector Space Models and Their Compositionality*. Association for Computational Linguistics.
- [32] Théo Trouillon, Johannes Welbl, Sebastian Riedel, Éric Gaussier, and Guillaume Bouchard. 2016. Complex Embeddings for Simple Link Prediction. In *Proceedings of the 33rd International Conference on Machine Learning*. PMLR, 2071–2080.
- [33] Chenxu Wang, Xin Wang, Zhao Li, Zirui Chen, and Jianxin Li. 2023. HyConvE: A Novel Embedding Model for Knowledge Hypergraph Link Prediction with Convolutional Neural Networks. In *Proceedings of the ACM Web Conference 2023*. 188–198.
- [34] Duo Wang, Yuan Zuo, Fengzhi Li, and Junjie Wu. 2024. LLMs as Zero-Shot Graph Learners: Alignment of GNN Representations with LLM Token Embeddings. *arXiv preprint arXiv:2408.14512* (2024).
- [35] Xiang Wang, Tinglin Huang, Dingxian Wang, Yancheng Yuan, Zhenguang Liu, Xiangnan He, and Tat-Seng Chua. 2021. Learning Intents behind Interactions with Knowledge Graph for Recommendation. In *Proceedings of The Web Conference 2021*. 878–887.
- [36] Zhen Wang, Jianwen Zhang, Jianlin Feng, and Zheng Chen. 2014. Knowledge Graph Embedding by Translating on Hyperplanes. In *Proceedings of the Twenty-Eighth AAAI Conference on Artificial Intelligence*. 1112–1119.
- [37] Yanbin Wei, Qiushi Huang, Yu Zhang, and James Kwok. 2023. KICGPT: Large Language Model with Knowledge in Context for Knowledge Graph Completion. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, 8667–8683.
- [38] Bishan Yang, Scott Wen-tau Yih, Xiaodong He, Jianfeng Gao, and Li Deng. 2015. Embedding Entities and Relations for Learning and Inference in Knowledge Bases. In *Proceedings of the International Conference on Learning Representations*.
- [39] Guangqian Yang, Yi Liu, Lei Zhang, Licheng Zhang, Hongtao Xie, and Zhendong Mao. 2024. Knowledge Context Modeling with Pre-trained Language Models for Contrastive Knowledge Graph Completion. In *Findings of the Association for Computational Linguistics: ACL 2024*. 8619–8630.
- [40] Yueji Yang, Divyakant Agrawal, H. V. Jagadish, Anthony K. H. Tung, and Shuang Wu. 2019. An Efficient Parallel Keyword Search Engine on Knowledge Graphs. In *Proceedings of the 35th IEEE International Conference on Data Engineering*. 338–349.
- [41] Liang Yao, Chengsheng Mao, and Yuan Luo. 2019. KG-BERT: BERT for Knowledge Graph Completion. *arXiv preprint arXiv:1909.XXXXX* (2019).
- [42] Liang Yao, Jiazhen Peng, Chengsheng Mao, and Yuan Luo. 2023. Exploring Large Language Models for Knowledge Graph Completion. *arXiv preprint* (2023). arXiv:2308.13916

- [43] Yichi Zhang, Zhuo Chen, Lingbing Guo, Yajing Xu, Wen Zhang, and Huajun Chen. 2024. Making Large Language Models Perform Better in Knowledge Graph Completion. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 233–242.
- [44] Yushan Zhu, Wen Zhang, Mingyang Chen, Hui Chen, Xu Cheng, Wei Zhang, and Huajun Chen. 2022. DualDE: Dually Distilling Knowledge Graph Embedding for Faster and Cheaper Reasoning. In *Proceedings of the Fifteenth ACM International Conference on Web Search and Data Mining*. 1516–1524.

A Model Training Procedure

The overall optimization pipeline of ReaLM is summarized in Algorithm 1. The procedure integrates structured KG semantics into a pre-trained LLM through a sequential multi-stage process. It begins with RotatE-based semantic embedding pretraining, followed by residual vector quantization to discretize entity embeddings into compact code sequences. The corresponding codewords are then added to the LLM vocabulary as new tokens, whose embeddings are initialized from the learned codebooks. During fine-tuning, ReaLM alternates between (i) updating the embeddings of newly introduced KG tokens and (ii) adapting the LLM internal parameters via LoRA modules. Ontology-guided class consistency is further applied during training to refine entity predictions and maintain semantic correctness. The detailed process is as follows.

Algorithm 1 Training Procedure of ReaLM

Require: KG $\mathcal{F}$, pretrained LLM, number of RVQ stages S , codebook size K , LoRA rank r , learning rates $\eta_{\text{token}}, \eta_{\text{LoRA}}$

Ensure: Fine-tuned LLM with integrated KG tokens

- 1: **Pretrain KG embeddings:** Train RotatE on $\mathcal{F}$ to obtain entity embeddings $\{e_h, e_t \in \mathbb{C}^d\}$
 - 2: **Residual Vector Quantization:**
 - 3: **for** each entity embedding $e \in \mathbb{C}^d$ **do**
 - 4: Map $e \rightarrow e_{\text{real}} \in \mathbb{R}^{2d}$
 - 5: Initialize residual $\text{res}_0 = e_{\text{real}}$
 - 6: **for** $s = 1$ to S **do**
 - 7: Select nearest codeword $c_{s,i_s} \in C_s$ minimizing $\|\text{res}_{s-1} - c_{s,i_s}\|$
 - 8: Update residual $\text{res}_s = \text{res}_{s-1} - c_{s,i_s}$
 - 9: **end for**
 - 10: Store quantized token sequence $[i_1, i_2, \dots, i_S]$
 - 11: **end for**
 - 12: **Initialize new token embeddings:** Sum codewords across stages to initialize $E([\text{CODE}_j]) = \sum_{s=1}^S c_{s,j}$
 - 13: **Extend LLM vocabulary:** Add K new KG tokens
 - 14: **Fine-tune LLM:**
 - 15: **while** not converged **do**
 - 16: Sample batch of triples (h, r, t) from $\mathcal{F}$ with dynamic candidate entity sampling
 - 17: Update embeddings of new KG tokens using gradient descent with learning rate η_{token}
 - 18: Update LoRA parameters A, B of LLM internal weights using gradient descent with learning rate η_{LoRA}
 - 19: Compute next-token prediction loss $\mathcal{L}_{\text{LLM}}$
 - 20: Apply ontology-guided class consistency to refine entity predictions
 - 21: **end while**
 - 22: **Return** fine-tuned LLM with integrated KG tokens
-

Data Construction. For each triple (h, r, t) , the head entity h is first quantized into a 32-dimensional integer code sequence $[i_1^h, i_2^h, \dots, i_{32}^h]$ through the residual vector quantization module described in Section 3.2. Similarly, every candidate entity $e \in \mathcal{E}$ (including the ground-truth tail t) is quantized into its corresponding code sequence $[i_1^e, \dots, i_{32}^e]$. These codes serve as discrete proxies for continuous entity embeddings and are directly mappable to the extended LLM vocabulary.

Each sample is then formatted as a structured prompt, combining natural language context and quantized codes:

Prompt: “Given the query triplet $(h, r, ?)$, the quantized code of h is $[i_1^h][i_2^h] \dots [i_{32}^h]$. Please predict the quantized code of the correct tail entity.”

The expected output sequence is the quantized code of the ground-truth tail entity t . This design ensures that the model must reason over both symbolic relations and quantized semantic representations, producing a structured prediction in the same discrete code space.

Tokenization and Vocabulary Extension. The tokenizer is extended to include a new set of code tokens $[\text{CODE}_1], [\text{CODE}_2], \dots, [\text{CODE}_K]$, corresponding to the K codewords in the residual vector quantization codebooks. Each token is assigned a fixed Token ID in the range $[128259, 129258]$, distinct from the native text token range $[1, 128258]$.

During tokenization, textual components (relations, instructions, entity names) are processed by the base LLM tokenizer, while each quantized integer $[i]$ is replaced by the corresponding $[\text{CODE}_i]$ token. This hybrid token stream thus maintains both linguistic and structural continuity:

$$[\text{Text}] + [\text{CODE}] + [\text{Text}] + [\text{CODE}] \dots$$

As shown in Figure 6, this unified token sequence allows the model to jointly encode semantic and symbolic information during training.

Fine-Tuning Objective. The fine-tuning process employs the standard next-token prediction objective:

$$\mathcal{L}_{\text{LLM}} = - \sum_{n=1}^N \log p(x_n | x_{<n}),$$

where each x_n denotes either a natural language token or a quantized code token. This objective implicitly aligns text and code modalities, teaching the model to interpret discrete quantized patterns as meaningful semantic representations of KG entities.

During training, all original LLM embeddings remain frozen to preserve the pretrained linguistic manifold, while embeddings of the newly introduced code tokens are trainable. Their initial values are set as the stage-summed codeword embeddings from the residual vector quantization module:

$$E([\text{CODE}_j]) = \sum_{s=1}^S c_{s,j},$$

ensuring that each token begins with semantically grounded initialization.

Parameter-efficient adaptation is achieved using Low-Rank Adaptation (LoRA) in the attention and feed-forward layers. Only the

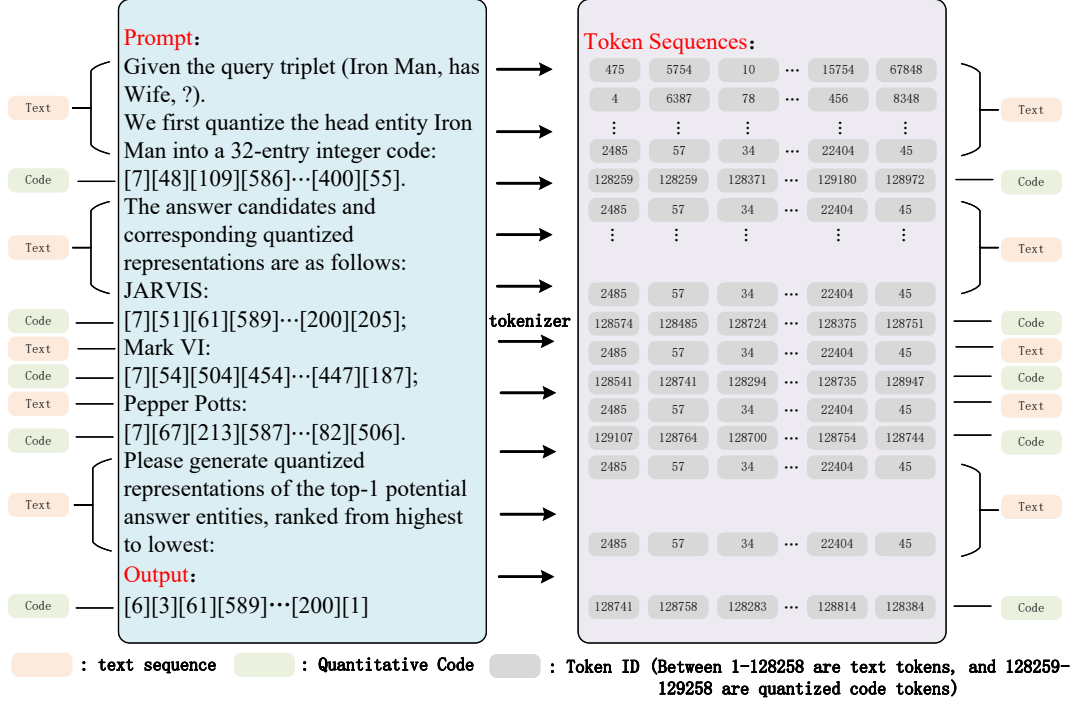


Figure 6: Tokenization process of LLM training corpus.

LoRA matrices A and B are updated, minimizing trainable parameters while maintaining generative fluency.

Training Configuration. Training is conducted using the AdamW optimizer with a learning rate of 1×10^{-3} , a per-device batch size of 4, and gradient accumulation of 4 steps, resulting in an effective batch size of 16. Mixed-precision training with bfloat16 (bf16) is employed to reduce memory consumption, and gradient checkpointing is enabled to further alleviate GPU memory usage. A linear warm-up is applied over the first 100 steps, followed by a cosine decay schedule. LoRA is applied to the q_proj and v_proj modules with a rank $r = 8$, $\alpha = 32$, and a dropout rate of 0.05, without updating bias terms. Training proceeds for up to 1,000,000 steps, with checkpoints saved every 10,000 steps. Candidate entities are sampled dynamically at each step to diversify relational contexts, and convergence is monitored via perplexity on a held-out validation set.

Inference and Output Decoding. At inference time, the model receives a prompt containing the quantized head code and relation description, and auto regressively generates the corresponding sequence of quantized code tokens. The generated index sequence $[i_1, i_2, \dots, i_{32}]$ is then decoded through the residual vector quantization codebooks to reconstruct the continuous embedding $\mathbf{e}_q$. The nearest neighbor in the pre-trained entity embedding space is selected as the predicted tail entity:

$$t^* = \arg \max_{t' \in \mathcal{E}} \cos(\mathbf{e}_q, \mathbf{e}_{t'}).$$

Through this training scheme, RealLM learns to seamlessly integrate discrete KG semantics into the generative space of LLMs,

aligning symbolic reasoning with linguistic comprehension under a unified next-token prediction paradigm.

B Training Setup and Computational Cost

All experiments are conducted using **two NVIDIA H100 GPUs**. While both GPUs are available for all runs, the training of each dataset is executed on a **single GPU**, and the second GPU is used only for parallel experiments or resource allocation. The fine-tuning procedure alternates between updating the embeddings of newly introduced KG-derived tokens and adapting the internal parameters of the LLM via LoRA. Mixed-precision training with bfloat16 (bf16) and gradient checkpointing are employed to reduce memory consumption and support long input sequences.

For the FB15K-237 dataset, the overall training process takes **approximately 187 hours**, including about **4 hours** for KGE pre-training, **6 hours** for residual vector quantization, and **176 hours** for LLM fine-tuning. For the WN18RR dataset, the total training time is **approximately 130 hours**, primarily due to its smaller scale and lower relational complexity. Model checkpoints are saved periodically, and training convergence is monitored using perplexity on a held-out validation set. Detailed computational costs and configurations will be released in the open repository.

C Dataset Description

In this work, we evaluate our proposed model on two ontology-enhanced KG datasets: **FB15K-237O** and **MN18RR-O**. Each dataset extends its original benchmark (FB15K-237 and WN18RR) by incorporating ontology-level information such as entity classes, relation

Table 4: Statistics of the datasets.

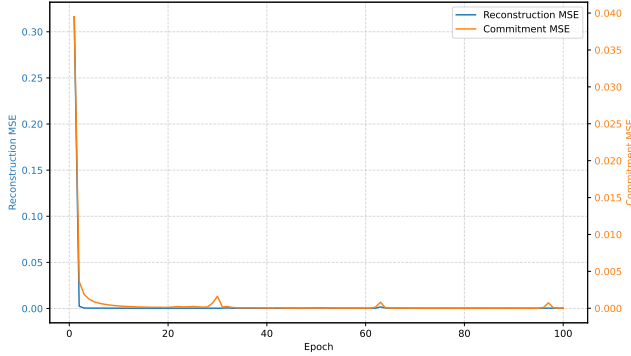
Dataset	Rel	Ent	Tri	Ont	Ont-cla	Ont-rel	Ont-com
FB15K-237O	237	14,541	310,116	18,763	16,073	93	2,597
MN18RRO	11	40,943	94,003	4,710	4,683	9	18

hierarchies, and logical constraints. These ontology annotations serve as symbolic priors to guide the reasoning process of model and to filter out inconsistent predictions. Detailed dataset statistics are presented in Table 4.

Specifically, **FB15K-237O** is derived from the Freebase KG and enriched with ontology information extracted from schema of Freebase and WordNet hierarchy. This includes type inheritance and relation composition rules that capture the semantic constraints among entities. **MN18RR-O**, on the other hand, extends WN18RR with hierarchical relations and lexical ontological mappings derived from WordNet, allowing the model to reason over both lexical and semantic dimensions of entities.

The ontology components (*Ont*, *Ont-cla*, *Ont-rel*, and *Ont-com*) respectively denote the number of ontology axioms, classes, relation hierarchies, and composition constraints incorporated into each dataset.

D Quantization Process Analysis

**Figure 7: Impact of codebook size on triple-classification performance.**

To investigate the convergence behavior of the quantization process, we monitored the reconstruction MSE and commitment MSE over 100 training epochs. The reconstruction error reflects how closely the quantized embeddings approximate the original continuous representations, while the commitment error measures the discrepancy between the encoder outputs and their assigned codewords, indicating the stability of codeword updates during optimization.

As illustrated in Figure 7, both metrics exhibit a rapid decline at the early training stage, followed by gradual stabilization as the model converges. The reconstruction MSE drops sharply within the first few epochs and then fluctuates slightly around 0.0003, while the commitment MSE decreases by several orders of magnitude to below 0.00001, confirming that the quantized representations

become highly consistent and well-aligned with the learned codebook. These results demonstrate that the proposed quantization framework achieves fast convergence and stable optimization, effectively balancing reconstruction fidelity and codeword compactness throughout training.

E Effect of Ontology Knowledge

Table 5: Comparison of entity-only, class-only, and ontology-constrained entity prediction (ReaLM) on WN18RR.

Model	MRR	Hits@1	Hits@3	Hits@10
ReaLM	0.608	0.575	0.615	0.699
Class-only Prediction	0.624	0.591	0.637	0.710
Entity-only Prediction	0.591	0.562	0.598	0.689

To isolate and quantify the contribution of ontology (class-level) information to KGC, we design a controlled experiment comparing three prediction paradigms: (1) **Entity-only Prediction**, where the model directly predicts target entities without ontology constraints; (2) **Class-only Prediction**, where the model predicts the class of the missing entity using the same quantization and tokenization pipeline as entities; and (3) **Ontology-constrained Entity Prediction (ReaLM)**, where entity predictions are refined by the predicted class, following the ontology-guided mechanism described in the main text.

The results in Table 5 show consistent performance differences among the three settings. The class-only model achieves the highest scores across all evaluation metrics, indicating that class prediction is statistically easier due to the much smaller and less sparse label space. This allows the model to learn stronger and more stable semantic priors.

In summary, these findings confirm that integrating ontology-level knowledge provides clear and measurable benefits for KGC. Predicting entity classes is inherently easier than predicting individual entities, and leveraging this class information to guide entity prediction consistently yields substantial improvements in accuracy and reliability. Importantly, these gains are achieved without modifying the underlying model architecture, further validating the practical effectiveness and robustness of ontology-guided reasoning.